Assignment 1 — How Matrix Size Affects a Basic Matrix Multiplication in Python, Java and C

Big Data · Grado en Ciencia e Ingeniería de Datos · ULPGC 2026-2027 · Individual Assignment 1 · Andrea Patruno · October 2026

Abstract. We compare the same triple-loop matrix multiplication (i-j-k order, float64, flat row-major arrays) in Python, Java and C, on the same shared inputs and the same machine, for n = 64 … 2048. For each configuration we take 5 timed repetitions of the kernel only and report median and interquartile range, together with the peak resident memory of the process. All three implementations produce bitwise identical results. C and Java finish n = 1024 in about 5–6 s, pure Python reaches only n = 512 (18.8 s) inside the 20 s budget. Runtime grows about as n^3 only while the matrices fit in cache: at n = 512 the time jumps ×34 (C) instead of ×8 because the column-wise access to B causes cache misses. Python is 44–250× slower than C, Java is 2.6× slower at n = 64 but equal from n = 512. The conclusions hold only for these implementations, settings and this machine.

1. Introduction and objective

Matrix multiplication is a basic Big Data kernel (graphs, machine learning, scientific computing) and its cost grows quickly with size. Before optimising it (later assignments) we need a controlled baseline.

Research question. How does the matrix size n affect the runtime and the memory use of the same basic triple-loop algorithm implemented in Python, Java and C?

- *Factor changed:* matrix size n (and the language).
- *Outcomes measured:* time per multiplication (median and IQR) and peak resident memory.

Hypotheses (stated before running).

- H1: the time grows as n^3 (×8 when n doubles) for all three languages.
- H2: C is the fastest, Java is close after JIT warm-up, Python is one to two orders of magnitude slower because it is interpreted.
- H3: C uses about $24n^2$ bytes for the three matrices plus a small runtime; Java adds the JVM (tens of MB); Python needs more memory per element because each float is an object.

2. Background

Algorithm. For C = A·B with n × n matrices, the straightforward method computes every C[i][j] as the dot product of row i of A and column j of B: three nested loops, n^3 multiply-add operations, $O(n^3)$ time and $O(n^2)$ memory [1].

Memory hierarchy. With row-major storage, the inner loop reads A[i][k] contiguously but B[k][j] with a stride of n·8 bytes, so each access to B can need a new cache line. When B no longer fits in the cache the cost per operation increases; with power-of-two strides many lines also map to the same cache sets (conflict misses) [2].

Languages. C is compiled ahead of time to machine code. Java is compiled to bytecode and then compiled at run time by the JIT, so the first runs are slower (warm-up). CPython interprets bytecode and each float is a separate heap object, so every operation in the inner loop costs much more than one machine instruction.

Floating-point error. A length-n dot product computed in floating point has error at most $\gamma_n \cdot \Sigma |a_k||b_k|$ with $\gamma_n \approx n \cdot u$, $u = 2^{-53}$ [3]. This is used to set the correctness tolerance.

3. Methodology

3.1 Implementations

The three programs (`src/`) contain the same loop, in the same order, on the same layout:

```
for i in 0..n-1:  for j in 0..n-1:  s = 0;  for k in 0..n-1:  s += A[i*n+k] * B[k*n+j];   C[i*n+j] = s
```

	Python	Java	C
Storage	flat `list` of Python floats	flat `double[]`	flat `double*` (malloc)
Numeric type	float64 (boxed objects)	double (primitive)	double
Execution	CPython 3.10.12 interpreter	OpenJDK 11.0.32, JIT (C2), `-Xmx2g`	GCC 11.4.0 `-O2 -std=c11`
Timer	`time.perf_counter_ns` [6]	`System.nanoTime` [7]	`clock_gettime(CLOCK_MONOTONIC)`

No library call replaces the loops. Unavoidable differences: Python stores pointers to float objects (≈ 32 bytes per element instead of 8) and checks types at every operation; Java checks array bounds; C uses `-O2` without `-ffast-math`, so the compiler cannot reorder the floating-point sum [8] and all three perform exactly the same additions in the same order.

3.2 Inputs

`scripts/gen_inputs.py` writes A and B once for every n as binary files (float64, little-endian, row-major), and the three programs read the same files. Generator: NumPy PCG64 [4], seed 42 + n, values uniform in [−1, 1); A is drawn first, then B.

3.3 Environment

Item	Value
Machine	MSI laptop, Intel Core i7-1255U (host: 10 cores / 12 threads), Windows host
Execution environment	Ubuntu 22.04.5 VM (kernel 6.8), **1 physical core / 2 logical processors** visible, 3.9 GB RAM
Caches seen by the VM	L1d 48 KiB, L2 1.25 MiB, L3 12 MiB
Threads	all three programs are single-threaded; no numerical library is used in the timed code

3.4 Correctness (before benchmarking)

`scripts/verify.py` runs the three programs on: a 2×2 case with known result, A·I = A and I·A = A (n = 50), A·0 = 0 (n = 50), and random matrices with n = 64 and n = 100 compared with NumPy `matmul` (reference only). Known-answer cases must match exactly. For random inputs $|a_{ik}|$, $|b_{kj}| \le 1$, so the error bound of §2 gives at most n^2u per entry; we use absolute tolerance $n^2 \cdot \text{eps}$ (eps = 2^{-52}, which also covers the reference's own error) and no relative tolerance.

3.5 Measurement protocol

- **Timing boundary:** kernel only. Loading the files and allocating A, B, C are outside the timer; the output array is allocated once and overwritten.
- **Repetitions:** 5 measured repetitions per configuration; warm-up runs excluded: Java 3 (JIT), C 1 and Python 1 (caches).
- **Short runs:** for n = 64 and 128 one sample times a batch of 64 and 8 multiplications, and reports the time per multiplication.
- **Result consumed:** after each sample the program sums all entries of C (checksum); the checksum is saved with every row.
- **Summary:** median, Q1 and Q3 (inclusive method), IQR = Q3 − Q1. All raw values are in `results/raw_times.csv`.
- **Memory:** peak resident set size (VmHWM in `/proc/self/status` [5]) of the process, read by the program itself after its last run. It includes the runtime (interpreter, JVM), so it is reported next to an estimate of the array storage: $3 \cdot 8n^2$ bytes for C/Java, $3 \cdot 32n^2$ for Python.
- **Sizes and budgets** (declared before running): n = 64, 128, 256, 512, 1024, 2048. Budget: 20 s for one multiplication, 2 GB peak memory. When 8 × the previous median exceeds 20 s, one multiplication is first attempted with a 20 s timeout; if it times out, larger sizes are not attempted.

4. Results

4.1 Correctness

All tests passed for the three implementations. The known-answer, identity and zero cases match exactly (error 0). Against NumPy the maximum absolute error is $4.4 \cdot 10^{-15}$ (n = 64, tolerance $9.1 \cdot 10^{-13}$) and $7.1 \cdot 10^{-15}$ (n = 100, tolerance $2.2 \cdot 10^{-12}$). The outputs of Python, Java and C are **bitwise identical** in every test, and the benchmark checksums are identical across languages for every n.

4.2 Runtime

Table 1 — Time per multiplication (seconds), median of 5 repetitions with IQR. "—" = not attempted after a timeout at a smaller size.

n	C median	C IQR	Java median	Java IQR	Python median	Python IQR	Java / C	Python / C
64	0.000097	0.000015	0.000256	0.000012	0.0246	0.0008	2.6	253
128	0.00154	0.00003	0.00231	0.00020	0.199	0.006	1.5	129
256	0.0126	0.0006	0.0183	0.0007	1.70	0.06	1.5	135
512	0.425	0.063	0.435	0.001	18.81	0.19	1.0	44
1024	5.88	0.55	5.24	0.62	timeout		0.9	—
2048	timeout		timeout		—			

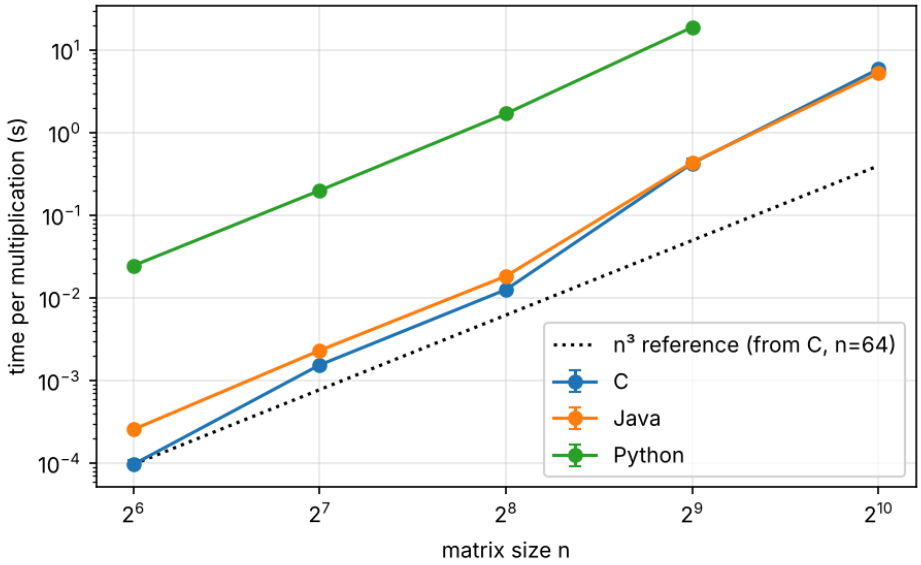


Figure 1 — Median time per multiplication vs n (log-log, error bars = IQR). The dotted line is n^3 scaled from C at n = 64.

Table 2 — Growth factor of the median when n doubles (n^3 model: ×8).

n → 2n	64→128	128→256	256→512	512→1024
C	15.9	8.2	33.8	13.8
Java	9.0	7.9	23.7	12.1
Python	8.1	8.5	11.1	—

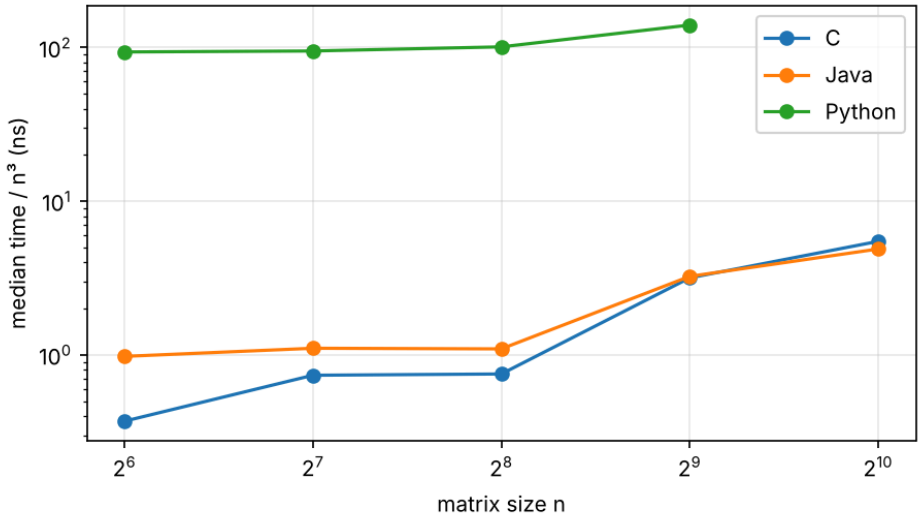


Figure 2 — Median time divided by n³: cost of one inner-loop iteration (one multiply-add) in nanoseconds.

Variability is small: the IQR is below 5 % of the median in most configurations; it is larger for C at n = 64, 512 and 1024 (15 %, 15 %, 9 %) and Java at n = 1024 (12 %).

4.3 Memory and limits

Table 3 — Peak resident memory of the process (MiB) and estimated storage of A, B and C (MiB).

n	C peak RSS	Java peak RSS	Python peak RSS	arrays C/Java (3·8n²)	arrays Python (≈3·32n²)
64	1.7	40.1	10.1	0.09	0.4
128	2.0	40.3	11.6	0.4	1.5
256	3.1	42.5	17.2	1.5	6
512	7.6	51.9	39.6	6	24
1024	25.6	82.6	—	24	—

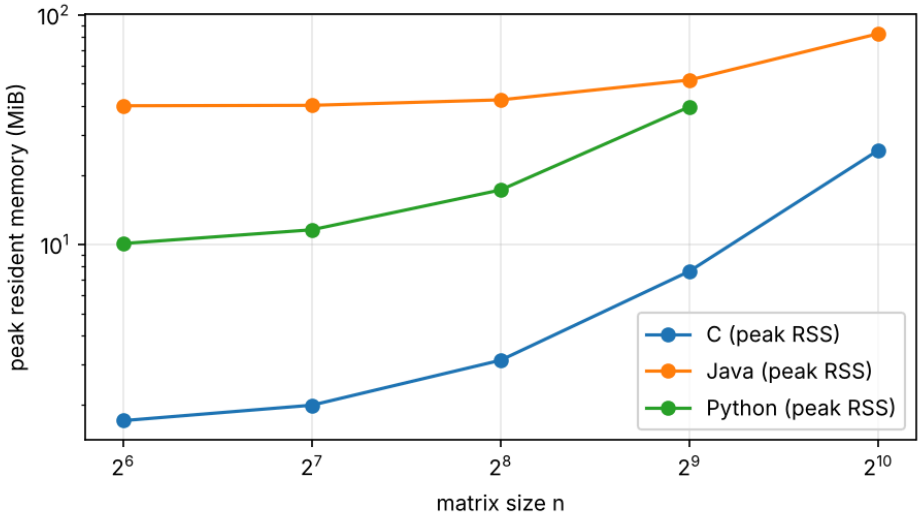


Figure 3 — Peak resident memory vs n (log-log).

Limits. Largest size completed inside the budget: n = 1024 for C and Java, n = 512 for Python. Timeouts (one multiplication not finished in 20 s, `results/limits.csv`): C at n = 2048, Java at n = 2048, Python at n = 1024. The memory budget (2 GB) was never reached; the largest attempted size, 2048, needs 96 MiB of arrays in C/Java, so the limit is time, not memory.

5. Discussion

H1 — cubic growth: only partly confirmed. Python grows almost exactly ×8 per doubling (8.1, 8.5) up to n = 256. C and Java grow ×8 only between 128 and 256; from 256 to 512 the time grows ×34 (C) and ×24 (Java). Figure 2 shows why: the cost of one multiply-add is constant only while the data stays in cache, then it rises from 0.37–0.75 ns (C, $n \le 256$) to 3.2 ns (n = 512) and 5.5 ns (n = 1024). At n = 512 one matrix is 2 MiB, larger than the L2 cache (1.25 MiB), and the inner loop jumps 4 KiB in B at every step: almost every access to B is a cache miss, and the power-of-two stride makes the lines compete for the same cache sets [2]. The first jump in C (×15.9 from 64 to 128) has the same cause on a smaller scale: at n = 64, B (32 KiB) fits in the 48 KiB L1, at n = 128 (128 KiB) it does not. The algorithm is $O(n^3)$, but the observed constant depends on the memory hierarchy. We did not measure cache misses with hardware counters, so this explanation is supported by the timing pattern and the cache sizes, not by direct cache measurements.

H2 — language ranking: confirmed only for small n. For $n \le 256$ C is fastest, Java is 1.5–2.6× slower and Python 129–253× slower. In Python every inner iteration executes several bytecode instructions, two list lookups and the creation of new float objects (≈ 94–101 ns per iteration against < 1 ns in C). For $n \ge 512$ the gap shrinks: Java equals C (0.435 s vs 0.425 s at n = 512; at n = 1024 Java's median is lower but the two IQRs overlap), and Python is "only" 44× slower. When the loop is limited by memory accesses, the quality of the generated code matters less. The extra cost of Java at n = 64 is plausibly due to array bounds checks and to short kernels where fixed overheads are more visible.

H3 — memory: confirmed. C's peak memory is the arrays plus ≈ 1.6 MiB of process (25.6 MiB at n = 1024 for 24 MiB of arrays). Java has a fixed JVM cost of ≈ 40 MiB, and its peak also includes the temporary byte buffers used to read the files (82.6 MiB at n = 1024). Python starts at ≈ 10 MiB and grows about 4–5× faster per element than C, because each value is a float object (24 bytes) referenced by a list slot (8 bytes); at n = 512 it uses 39.6 MiB against 7.6 MiB for C.

Limitations.

- One machine and one VM with a single physical core: the hypervisor and the Windows host can add noise (larger IQR in some C runs). The results do not describe other machines.
- Peak RSS includes runtime and file-loading buffers; it is not the same quantity as array storage, so the two are reported separately.
- Only 5 repetitions per configuration; the timeouts give a time limit, not a measured time.
- The batching for $n \le 128$ means that those samples reuse warm caches.
- The cache explanation is not verified with hardware counters (e.g. `perf`).

6. Conclusions

On this machine, with the same algorithm, inputs and arithmetic, the triple-loop multiplication grows as n^3 only while the matrices fit in cache; beyond that, memory access dominates and the growth per doubling is much larger than 8. C and Java reach n = 1024 in about 5–6 s and become equivalent for large n; pure Python is 44–250× slower and reaches only n = 512 in the 20 s budget. Memory follows the $8n^2$ (C, Java) or $\approx 32n^2$ (Python) storage plus a fixed runtime cost. These results are limited to these implementations, compiler and runtime settings, and hardware; they do not show that one language is faster in general. The loop order that reads B by columns is the main bottleneck, which is the starting point for the optimisations of Assignment 2.

7. References and materials

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[4] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[5] Linux man-pages project, “proc(5) — Linux manual page.” [Online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html

[6] Python Software Foundation, “time — Time access and conversions,” Python 3.10 documentation. [Online]. Available: https://docs.python.org/3.10/library/time.html

[7] Oracle, “System (Java SE 11 & JDK 11),” API documentation. [Online]. Available: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/System.html

[8] Free Software Foundation, “Options that control optimization,” GCC online documentation. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

Materials. GitHub repository: https://github.com/andreapatruno1/Assignment1_BD. It contains the source code, the README with exact build and run commands, the input generator, raw measurements (`results/raw_times.csv`), correctness results, environment description and the scripts that produce all tables and figures (`scripts/analyze.py`).

Use of generative AI. Generative AI was used to draft code, scripts and report text; the author ran and checked all experiments.